

[43] Neighbor-sensitive hashing

저자: Y. Park, M. Cafarella, and B. Mozafari **학술지:** Proc. VLDB Endow., vol. 9, no. 3, p. 144-155, 2015 **주제:** 이웃 감지 해싱(NSH) - 이웃 정보를 활용한 새로운 해싱 기법

요약

본 논문은 Neighbor-Sensitive Hashing(NSH)라는 새로운 해싱 기반 ANN 기법을 제시한다. 전통적인 LSH와 달리, NSH는 각 데이터 포인트의 이웃 구조를 명시적으로 활용하여 해시 함수를 설계한다.

NSH의 핵심 아이디어는 "이웃들이 같은 해시 버킷에 들어갈 확률을 높인다"는 것이다. 이를 위해 실제 데이터로부터 학습된 이웃 그래프를 기반으로 최적화된 해시 함수를 구성한다.

본 논문과의 관계: Exqutor의 필터링 기반 검색에서 필터 조건을 만족하는 벡터들의 이웃 구조를 활용하면, NSH 처럼 필터 친화적인 인덱싱을 구성할 수 있다.

상세분석

43.1 LSH의 한계와 NSH의 동기

LSH의 문제점:

LSH는 무작위 함수를 사용:

- 데이터 분포 미고려
- 실제 이웃 관계 무시
- 높은 거짓양성율

문제:

- 벡터 A와 B가 유사하지만
- LSH 함수의 무작위성으로 인해
- 다른 해시 버킷에 들어갈 수 있음

NSH의 아이디어:

데이터의 실제 이웃 구조를 분석:

1. 데이터로부터 이웃 그래프 구성
2. 이웃 관계를 유지하는 해시 함수 설계
3. 유사한 점들이 같은 해시 버킷에 들어가도록 학습

결과: LSH보다 정확하고 효율적인 검색

43.2 이웃 그래프 구성

KNN 그래프:

각 점 p 에 대해:

1. 모든 다른 점과의 거리 계산
2. 가장 가까운 k 개 선택
3. $p \rightarrow \{k\text{개 이웃}\}$ 연결

예 ($k=3$):

점 A: [B, C, D] (A와 가장 가까운 3개)

점 B: [A, E, F]

점 C: [A, D, G]

...

그래프 특성:

방향 그래프:

- A가 B의 이웃이라고 해서 B가 A의 이웃은 아닐 수 있음

상호 이웃 (mutual neighbors):

- A와 B가 서로를 이웃으로 포함하는 경우
- 더 강한 유사도 신호

계산 복잡도:

KNN 그래프 구성: $O(n^2)$

(모든 점 쌍의 거리 계산)

대규모 데이터: 근사 KNN 그래프 사용

- 샤할링 또는 클러스터링으로 근사
- $O(n \log n)$ 또는 $O(n\sqrt{n})$

43.3 NSH 함수 설계

학습 문제로의 재정의:

목표: 해시 함수 h 를 설계하여
이웃인 점들은 같은 해시 값 확률 높음
다른 점들은 다른 해시 값 확률 높음

제약:

- 해시 값은 $0 \sim m-1$ 범위 (m 개 버킷)
- 계산 비용 낮아야 함

최적화 공식:

최대화:

$$\sum_{\{(i,j) \in \text{edges}\}} [h(i) = h(j)]$$

(이웃 쌍이 같은 버킷)

최소화:

$$\sum_{\{(i,j) \notin \text{edges}\}} [h(i) = h(j)]$$

(비이웃 쌍이 같은 버킷)

43.4 NSH의 구체적 구현

방법 1: Spectral Hashing (스펙트럼 해싱)

그래프 라플라시안 계산:

$$L = D - A$$

(D : 차수 행렬, A : 인접 행렬)

라플라시안의 고유벡터 사용:

$v_1, v_2, \dots$ (가장 작은 고유값부터)

해시 함수:

$$h(x) = [\text{sign}(v_1^T x), \text{sign}(v_2^T x), \dots]$$

(이진 해시 코드 생성)

결과: 각 비트가 이웃 보존성 최적화

방법 2: 최소 컷 기반 (Min-cut)

목표: 이웃을 최대한 같은 그룹으로
비이웃을 다른 그룹으로

알고리즘:

1. 그래프에서 최소 컷 찾기
2. 이웃은 같은 쪽, 비이웃은 다른 쪽
3. 각 컷이 하나의 비트를 정의

결과: 이웃 관계를 명시적으로 보존하는 해시

방법 3: 확률적 접근

마르코프 연쇄 몬테카를로(MCMC) 사용:

1. 현재 해시 할당 상태에서 시작
2. 점진적으로 할당 개선
3. 수렴 시 최적 또는 근최적 해시

장점: 이론적 보장 가능

단점: 계산 비용 높음

43.5 다중 해시 테이블과 검색

k개 해시 함수 사용:

```
함수 1:  $h_1(x) = [\text{bit}_{0\_1}, \text{bit}_{1\_1}, \dots]$   
함수 2:  $h_2(x) = [\text{bit}_{0\_2}, \text{bit}_{1\_2}, \dots]$   
...  
함수 k:  $h_k(x) = [\text{bit}_{0\_k}, \text{bit}_{1\_k}, \dots]$ 
```

각 함수는 이웃 구조의 다른 측면 캡처

검색 알고리즘:

```

procedure NSH_SEARCH(query_q, K, k_tables):
    all_candidates = []

    for hash_function_idx = 1 to k_tables:
        h_idx = hash_function_idx

        // 쿼리의 해시 코드 계산
        query_hash = compute_hash(q, h_idx)

        // 같은 해시 코드의 모든 점 찾기
        bucket_contents = hashtable[h_idx][query_hash]
        all_candidates.extend(bucket_contents)

    // 중복 제거
    all_candidates = unique(all_candidates)

    // 거리 기반 정렬
    candidates_with_dist = [
        (c, distance(q, c)) for c in all_candidates
    ]

    // 가장 가까운 K개 반환
    return sorted(candidates_with_dist,
                  key=lambda x: x[1])[:K]

```

43.6 이론적 보장

성능 분석:

이웃 보존율 (Recall):

$$= (\text{검색된_이웃_개수}) / (\text{실제_이웃_개수})$$

정밀도 (Precision):

$$= (\text{실제_이웃}) / (\text{검색된_점_개수})$$

NSH의 이론:
 적절한 k_tables 선택 시
 높은 recall과 precision 달성 가능

시간 복잡도:

검색: $O(k_tables \times \text{평균_버킷_크기} + \text{정렬})$

대부분의 경우:

$O(k_tables \times (\text{평균_버킷_크기}))$

평균 버킷 크기 $\ll n$

43.7 NSH vs LSH vs HNSW

지표	LSH	NSH	HNSW
함수 구성	무작위	학습 기반	N/A
정확도	중간	높음	매우 높음
속도	빠름	빠름	매우 빠름
메모리	중간	중간	낮음
인덱싱 비용	낮음	중간	높음
파라미터 조정	복잡	간단	간단

43.8 필터링과의 통합

필터 조건을 고려한 NSH:

문제:

1. 필터를 만족하는 점들의 부분집합으로 이웃 그래프 구성
2. 해시 함수는 필터 만족 점들 사이의 이웃 보존

알고리즘:

```
// 단계 1: 필터된 KNN 그래프 구성
filtered_points = {p | filter_predicate(p)}
knn_graph = build_knn_graph(filtered_points, k)

// 단계 2: NSH 함수 학습
hash_functions = learn_nsh_functions(knn_graph)

// 단계 3: 검색
query_candidates = search_with_hash_functions(q)
return filter_results_by_distance(
    query_candidates, K
)
```

성능 특성:

필터 선택도 높음 (많은 점 선택):

- 필터 없는 NSH와 유사
- 이웃 구조 더 풍부
- 좋은 성능

필터 선택도 낮음 (적은 점 선택):

- 이웃 구조 희소
- 해시 함수 학습 어려움
- 성능 저하 가능

43.9 실제 구현 고려사항

이웃 그래프 유지:

- 정적 데이터: 한 번만 구성
동적 데이터: 주기적 재구성 필요
- 시간 비용 높음 ($O(n^2)$)
 - 근사 기법 필요

해시 함수의 저장:

- k개 해시 함수 저장:
- 스펙트럼 해싱: k개 벡터 (고유벡터)
 - 비트맵: 각 함수당 n비트
 - 메모리: $O(k \times n)$ 비트

캐시 최적화:

- 이웃 쌍들이 자주 함께 접근:
- 캐시 미스 감소
 - 메모리 대역폭 효율성 향상
 - 전체 검색 속도 15~30% 향상

43.10 동적 NSH 구성

증분 업데이트:

새로운 점 x 추가:

1. 기존 이웃 그래프에서 x 의 k 개 이웃 찾기
2. 기존 점들의 이웃도 잠재적으로 변경
3. 영향받는 해시 함수 부분 재학습

전체 재구성보다 빠름:

시간: $O(k \times n)$ vs $O(n^2)$

43.11 필터와 이웃 구조의 관계

필터-유도 클러스터링:

벡터 공간의 이웃 관계와

필터 조건의 상관성 분석:

예:

"category=전자제품" 필터

→ 전자제품 벡터들이 서로 이웃일 확률

만약 높은 상관성:

- 필터 조건별 별도 NSH 인덱스 구성
- 각 인덱스가 더 효율적

낮은 상관성:

- 필터와 무관하게 전체 NSH 사용
- 필터링은 별도로 수행

43.12 NSH의 실제 성능 (논문 기준)

데이터: GIST, SIFT 벡터들 (백만 개)

회수율 (Recall):

NSH: 97~99%

LSH: 85~90%

HNSW: 98~99%

쿼리당 시간:

NSH: 2~5ms

LSH: 3~8ms

HNSW: 0.5~2ms

메모리:

NSH: 2GB

LSH: 2.5GB

HNSW: 3GB

추가 제기 문제

1. **필터 조건하에서의 이웃 구조**: 특정 필터 조건을 만족하는 벡터들의 이웃 구조가 전체 벡터의 이웃 구조와 어떻게 다를까?
2. **다중 필터와 NSH**: 여러 속성에 대한 필터가 있을 때, 각 필터 조합에 대해 별도의 NSH를 유지할 경우의 메모리 비용은?
3. **이웃 그래프의 동적 업데이트**: 필터 조건이 자주 변할 때, 이웃 그래프를 효율적으로 유지할 수 있을까?
4. **필터 최적 해시 함수**: NSH 학습 시 필터 조건 정보를 미리 제공하면, 필터 친화적인 해시 함수를 설계할 수 있을까?
5. **필터 선택도와 이웃 구조의 희소성**: 필터 선택도가 낮을 때(적은 벡터 선택), 이웃 구조의 희소성으로 인한 NSH 성능 저하를 어떻게 보완할 것인가?
6. **계층화된 필터와 이웃**: 필터 조건들의 계층(예: category > subcategory > item_type)을 이용하여 이웃 구조를 더 효율적으로 활용할 수 있을까?
7. **실시간 필터 변경**: 사용자가 쿼리 중 필터 조건을 변경할 때, 사전 학습된 NSH를 조정하거나 캐싱하는 전략은?
8. **필터 친화적 이웃 정의**: 거리 기반이 아닌 필터 조건 기반의 이웃을 정의하여 NSH를 구성할 수 있을까?